WasteWatch — Approaches

Six Design Approaches for AI-Driven Dumping Detection, End-to-End
Including Phone-Call Escalation Platform Options

Prepared for: internal engineering review
Version: August 2026
Prepared by: Satyaki Solutions Pvt Ltd

Table of Contents

1. Executive Summary
2. Approach 1 — Simple Trigger Swap + Narration Review Queue committed
3. Approach 2 — Full ROI-Gated Temporal State Machine chosen direction
4. Approach 3 — Real Hardware Validation Findings measured
5. Approach 4 — Inference Speed-Up Options (vLLM-omni / TensorRT / Triton)
6. Approach 5 — Fast-Detector Trigger Gating
7. Approach 6 — Per-Object Tracking with Occlusion-Aware Persistence
8. Call Escalation Platform Options (incl. Jambonz)
9. Synthesis: How the Six Approaches Compose Into One System
10. Open Decisions

1. Executive Summary

This document consolidates six distinct design approaches explored over the course of investigating how to upgrade WasteWatch's illegal-dumping detection with a vision-language model (MiniCPM-o 4.5), plus a seventh section evaluating phone-call escalation platforms including Jambonz. These are not six competing alternatives to pick one from — they are six layers of the same investigation, each answering a different question that the previous one raised. Read in order, they tell a coherent story: from "what's the smallest change we can ship" (Approach 1), to "what's the full production architecture" (Approach 2), through "does it actually work on our hardware" (Approach 3, which found a real latency problem), to three different ways of solving that problem (Approaches 4–6), each addressing a different weakness the previous one exposed.

The single most important finding across all six: **a vision-language model run continuously and naively is too slow (~1.8 seconds per frame, measured on real hardware) to serve as the primary detection mechanism at any useful frame rate.** Every approach after Approach 3 is, in one way or another, a response to that fact — either by making the model faster (Approach 4), by calling it less often via a cheap trigger (Approach 5), or by making the underlying detection logic sharper so fewer, better-timed VLM calls are needed at all (Approach 6).

The report closes with a call-platform comparison (Jambonz vs. the already-partially-built SIP/Twilio/Asterisk gateway) and a synthesis chapter showing how these pieces are intended to fit into one final architecture, followed by the decisions still open and unresolved as of this writing.

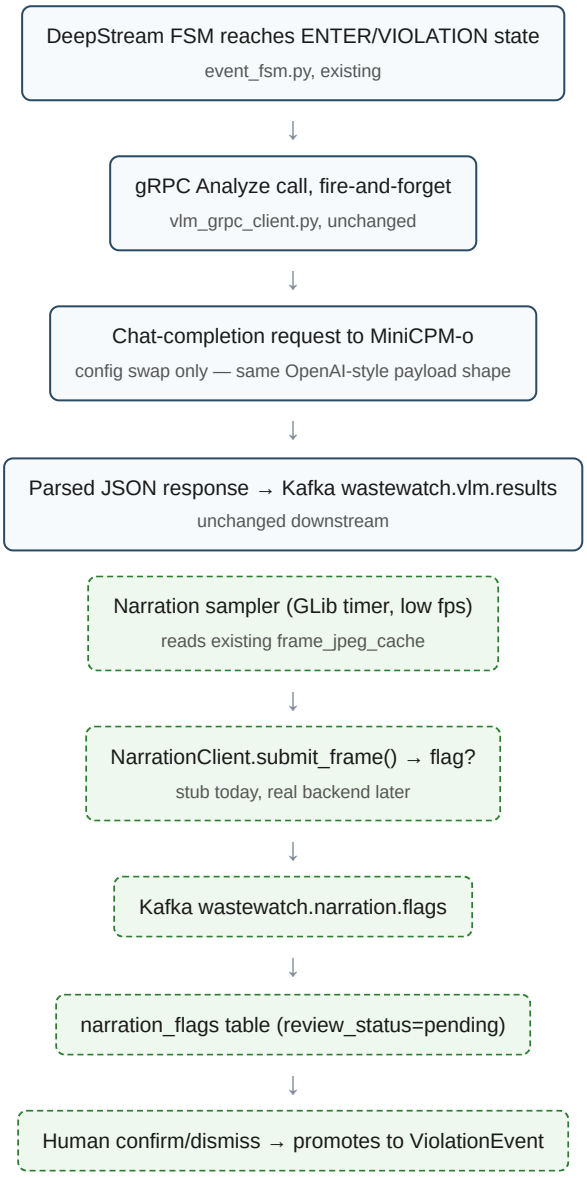
2. Approach 1 — Simple Trigger Swap + Narration Review Queue

committed to git

What it is

The smallest possible upgrade: swap the model behind the existing trigger-based VLM call from Qwen2.5-VL-7B-AWQ to MiniCPM-o 4.5, with no other architectural change. Separately, add a basic always-on narration capability that free-narrates each camera's scene at low frequency and drops any flagged anomaly into a human review queue, rather than an automatic alert.

End-to-end flow



Why it exists

This was the starting point before any of the bigger architecture was discussed — a low-risk, fast, fully-scoped change that could ship independently of everything else in this document. It required no new database infrastructure beyond one small table, no rewrite of existing services, and no hardware validation before committing to a plan.

Status

Design spec and a full TDD implementation plan were written and committed to the repository (`docs/superpowers/specs/2026-08-21-minicpmo-vlm-design.md` and its accompanying plan document). This approach has since been superseded in scope by Approach 2, at the user's explicit direction ("Approach A — Replace"), but the artifacts remain in the repository as the smaller, previously-approved alternative.

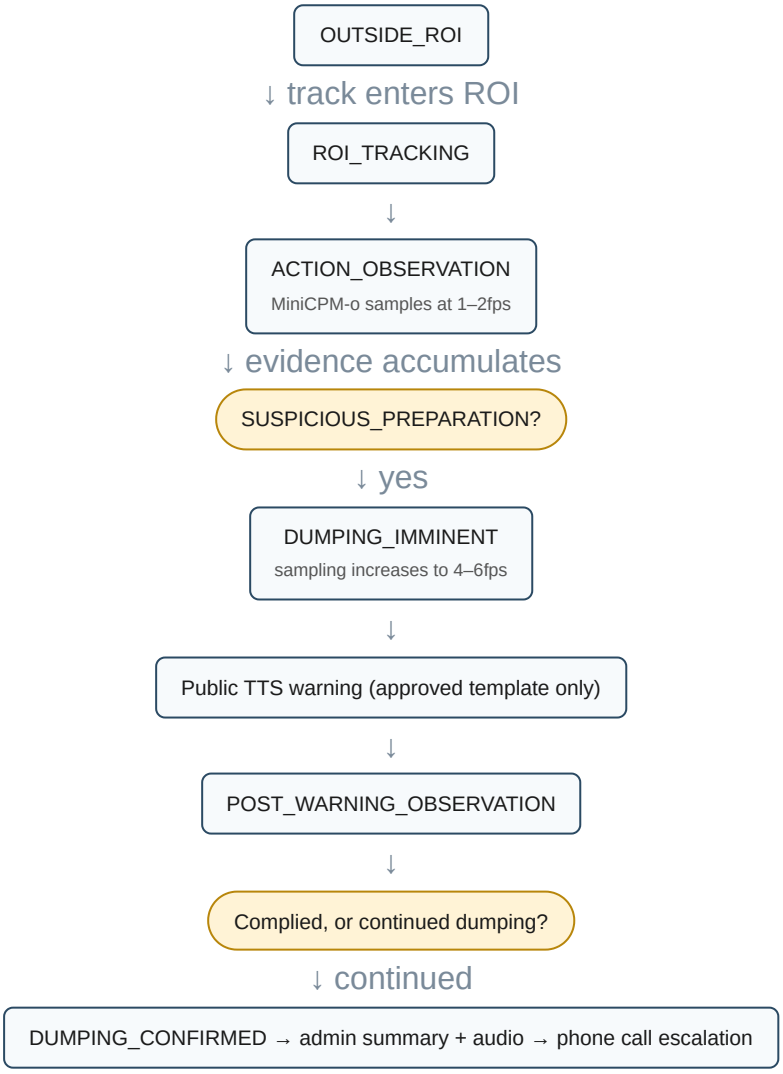
3. Approach 2 — Full ROI-Gated Temporal State Machine

chosen direction, Phase 0 done

What it is

A complete production architecture, specified by the user in full detail: person/vehicle detection and tracking (existing), a per-track state machine that only begins semantic monitoring once a relevant subject enters a configured ROI, temporal (multi-frame) action understanding via MiniCPM-o, a rule-based risk-scoring engine that combines deterministic and semantic signals over time, a safe-template public warning system, post-warning compliance monitoring, an administrator text-and-audio summary, and phone-call escalation with acknowledgement handling — all with full incident persistence for audit purposes.

End-to-end flow



Recovery transitions

The design explicitly allows evidence to weaken: `SUSPICIOUS_PREPARATION → ACTION_OBSERVATION`, `DUMPING_IMMINENT → ACTION_OBSERVATION`, and any active state can transition to `EXITED` once the tracked subject leaves. Ordinary activity — walking through, parking, deliveries, waiting — must never alert on ROI entry alone; only accumulated temporal evidence should escalate state.

Phase 0 findings (real repository state, verified before any code was written)

Area	Status found
ROI + tracking	Exists and reusable (`event_fsm.py` , `nvdsanalytics` polygon filter)
Vehicle-interaction / carried-object / persistence signals	Do not exist anywhere — genuinely new logic required
Redis	Already properly wired (throttle, escalation, health) — reusable as-is
SIP/Twilio	`SIPGateway` already implements 3 call backends + an escalation ladder, but has no DTMF acknowledgement, no idempotency, and no persisted call records
TTS	**Direct conflict** — current engine hardcodes an aggressive, shaming, all-caps tone with a fixed accusatory wrapper; the new spec's safe-template policy requires rewriting this live behavior, not extending it
Object storage	Does not exist — evidence is local disk + Dropbox only; the spec's asset-storage abstraction is new infrastructure
Event schema versioning	Does not exist — every Kafka payload today is an unversioned flat dict
Baseline tests	Pipeline: 7/7 pass. Backend: uncollectable due to a pre-existing, unrelated missing dependency

> **Risk flagged at Phase 0:** this is a genuinely multi-week build, not a single implementation pass. Several of its assumed dependencies (object storage, event versioning, safe-TTS templates) do not exist in the codebase today and must be built, not adapted.

Status

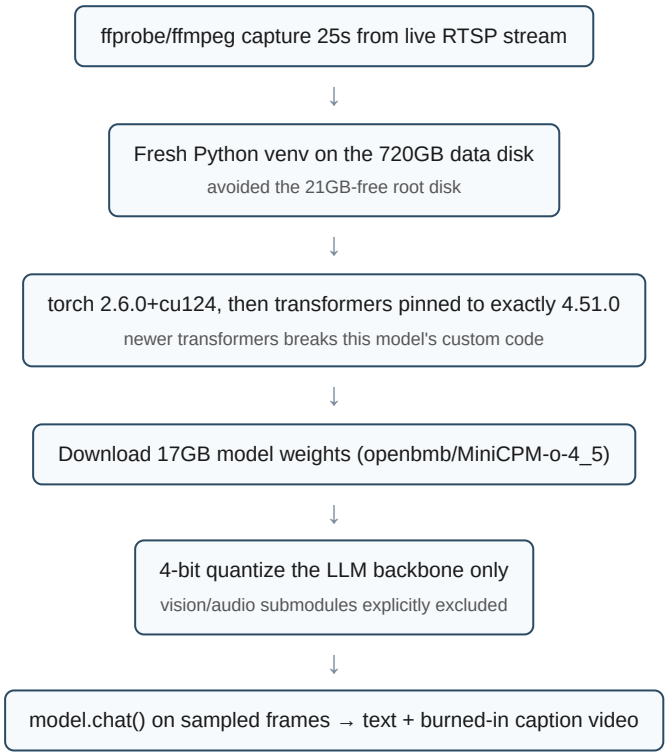
Confirmed by the user as the chosen direction ("Approach A — Replace"), superseding Approach 1's smaller scope. Phase 0 (repository inspection, baseline test run) is complete. Implementation has not yet started — real-world hardware validation (Approach 3) was done first, and it surfaced a fundamental latency problem that the remaining approaches in this document (4, 5, 6) all respond to.

4. Approach 3 — Real Hardware Validation Findings measured on real hardware

What was actually done

Before committing further engineering effort to Approach 2, a live end-to-end test was run: a 25-second clip was captured from a real camera (`rtsp://192.168.1.8:8654/press-brake-01`), MiniCPM-o 4.5 was installed from scratch on the actual target GPU, and inference was run on the captured frames, producing both a text description and a captioned output video.

End-to-end flow of the test itself



What the model actually produced

> The model correctly identified two workers at an industrial press-brake, described their clothing (brown jacket/yellow gloves/black beanie; dark jacket/black beanie), the specific actions (one operating machine controls, one loading a metal sheet), the environment, and explicitly assessed no unsafe behavior visible — in a factual, non-accusatory tone. This is qualitatively exactly the safe-descriptor style Approach 2's public-warning policy requires.

The critical measured number

Metric	Measured value
Model load time (one-time)	382 seconds (~6.4 minutes)
First inference call (CUDA warmup)	19.5 seconds
Steady-state single-frame latency	**~1.8 seconds/frame** (range 1.4–2.1s)
Peak VRAM used	8.09 GB (of 12 GB available)

> **This is the single most consequential finding in this document.** A 1fps target requires under 1.0s/frame; the measured throughput is roughly 0.55fps. A 3fps target requires under 0.33s/frame — over 5x too slow. This

> measurement used the simple, naive HF `transformers.chat()` call, not a proper serving engine — that distinction is exactly what Approach 4 investigates.

Secondary finding: real-world install friction

Getting to this measurement required resolving, one at a time, eight separate missing/incorrect dependencies (`librosa`, `soundfile`, the correct `minicpmo-utils` package name, `einops`, `onnx`, `onnxruntime`, `hyperpyyaml`, `diffusers`, `torchaudio`, `torchvision`) plus one hard version pin (`transformers==4.51.0` exactly) and a 4-bit-quantization module-selection bug specific to this custom architecture. None of this was resolvable by reading the model card alone — it required iterative trial. This confirms the "vllm-omni/MiniCPM-o serving immaturity" risk flagged during the original design research was not theoretical.

5. Approach 4 — Inference Speed-Up Options

Three technical paths were discussed to close the latency gap found in Approach 3.

Option A: Triton Inference Server

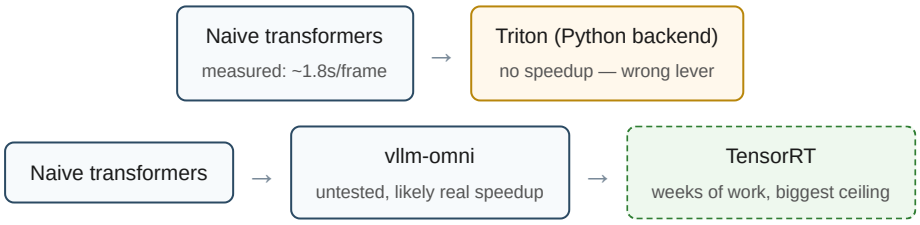
> Triton is a serving/batching/routing layer, not a compute accelerator. Wrapping the same slow `transformers.generate()` call in Triton's Python backend gives zero speedup — it only helps when paired with an already-fast backend (a TensorRT engine, or a proper LLM serving engine). Ruled out as a standalone fix.

Option B: TensorRT

Would give a real speedup, but is a far bigger job for this model than it is for the repository's existing RF-DETR detector. RF-DETR is a standard architecture with established ONNX/TensorRT export tooling. MiniCPM-o 4.5 is a custom architecture (SigLip2 vision encoder + custom resampler + Qwen3-8B backbone, all wired together in bespoke `modeling_minicpmo.py` code) with no existing public TensorRT conversion path. Each submodule would need hand export to ONNX, dynamic-shape handling (variable frame counts, variable resampler slicing), and custom engine building — realistically weeks of work with real risk that custom attention patterns fail to export cleanly at all.

Option C: vLLM / vllm-omni (recommended next step, not yet tested)

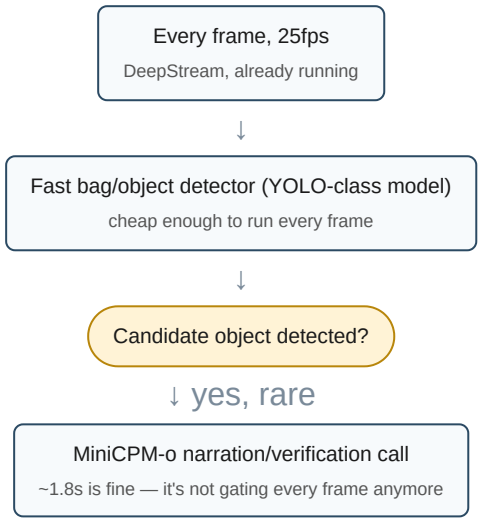
vLLM's speedup comes from continuous batching, paged attention, and optimized CUDA kernels for the LLM decode step — the same category of win as TensorRT, but with an existing, vendor-documented path for this exact model family (`vllm-omni`), rather than building a conversion pipeline from scratch. This has not yet been measured; every latency number in this document used the naive `transformers` path.



6. Approach 5 — Fast-Detector Trigger Gating

What it is

Rather than making the VLM itself faster, avoid needing it to run continuously at all. A cheap, fast, purpose-trained (or off-the-shelf) detector watches every frame instead, and only when it flags a relevant object (e.g. a bag) does the system call MiniCPM-o — at that point, the 1.8s/call latency is irrelevant, because it's no longer competing against a real-time per-frame budget, only against how quickly a warning needs to fire after a bag is spotted (seconds, not fractions of a second).



Why this is the same pattern as the existing repository

This is exactly the "deterministic trigger + selective VLM" pattern already used for the current trigger-based alert path (FSM state change → VLM call), and matches the evidence-signal list already present in Approach 2's design (carried object, object-ground contact, etc.) — just formalized as a trained detector instead of a heuristic.

What it requires

1. **Labeled data** — no bag-labeled dataset exists yet from this camera environment; this is the real blocker for a custom-trained model
2. **Model choice** — RF-DETR (already deployed) is detection-only; a segmentation head or a second lightweight model (YOLOv8/11-seg class) would run alongside it
3. **Trigger logic** — bag detected + person/vehicle nearby in ROI fires the same style of trigger call already used today

Status

Discussed and directionally agreed as correct; a zero-shot open-vocabulary detector (YOLO-World, chosen over Grounding DINO for its real-time design) was proposed as a fast way to validate the trigger-gating architecture before investing in custom training or data collection — installation was started but paused mid-decision as the user began exploring the distillation idea (Approach 6's data question) and then the deeper object-tracking analysis in Approach 6 below. Not yet built.

7. Approach 6 — Per-Object Tracking with Occlusion-Aware Persistence

What triggered this approach

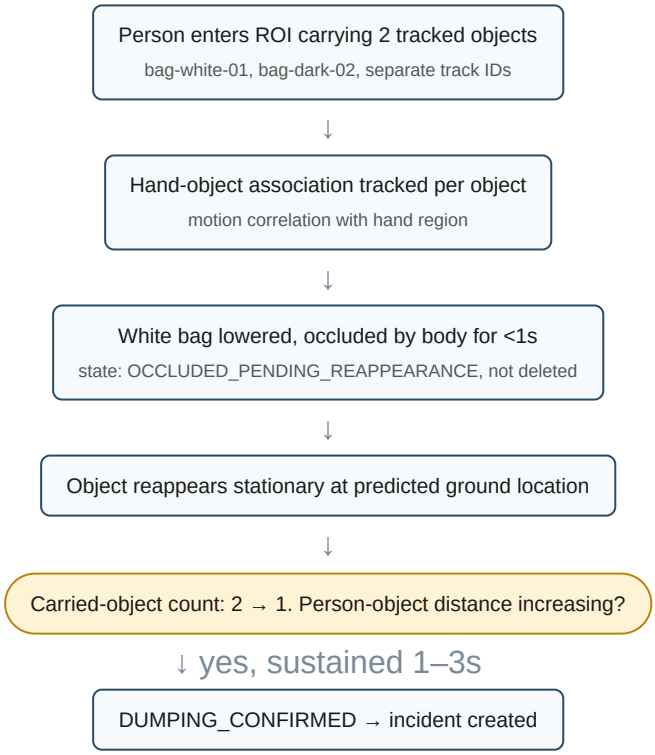
A detailed frame-by-frame analysis of a real sample clip (a person carrying two bags near a tree-side dumping ROI) revealed that the obvious detection rule — "an object was left behind" — misses this specific, realistic dumping pattern: the white bag is lowered, briefly hidden behind the person's body for well under a second, and released without any visible throwing motion, after which the person continues walking with only the remaining dark bag. A captioning system without per-object tracking would correctly but uselessly report "a person is walking beside a tree carrying bags," missing the event entirely.

The correct signal, precisely stated

```
carried-object count decreases
+ one object becomes stationary
+ person-object separation distance increases
+ person does not return to retrieve it
```

This combination — not "object left behind" alone — is the reliable dumping signal for this pattern, and it requires tracking each carried object individually, not just the person.

End-to-end flow for this pattern



Required new tracking capabilities (none of which exist in the repository today)

- **Small carried-object tracking** — tracking bags/containers as distinct objects, not just people
- **Hand-object association** — whether an object moves correlated with a person's hand/wrist region
- **Object inventory change** — continuously comparing carried-object count before vs. after an action
- **Occlusion-aware persistence** — do not delete a track when temporarily hidden by a person's body, a passing vehicle, or another pedestrian; hold it as pending and search near its predicted location
- **Static-object verification** — confirm an object remains in place after the person departs

Revised model output shape

```
{
  "subject_track_id": "person-104",
  "object_event": {
    "object_track_id": "bag-white-01",
    "event": "released",
    "release_location": "tree-base dumping ROI",
    "person_object_distance_increasing": true,
    "object_stationary": true,
    "persistence_seconds": 2.4
  },
  "remaining_carried_objects": ["bag-dark-02"],
  "recommended_state": "DUMPING_CONFIRMED"
}
```

Relationship to Approaches 2 and 5

This is not a replacement for Approach 2's state machine — it is a refinement of it, extending per-*track* monitoring to per-*object* monitoring, with the object-inventory-change signal added to the risk engine's existing evidence-signal list. It also strengthens the case for Approach 5's trigger-gating: the fast detector's job becomes not just "is there a bag" but "did the carried-object count just change," which is an even cheaper, more specific signal to compute continuously than running a VLM.

Status

Fully specified as an analysis, not yet folded into the committed design spec at the user's explicit instruction ("just have this context... nothing to work on yet"). Captured here for the record.

8. Call Escalation Platform Options

Current state (from Phase 0 findings)

`backend/app/alerts/sip_gateway.py` already implements a working, if incomplete, call escalation system: a `SIPGateway` class with three backends tried in priority order (local `pjsua2` PBX registration, Asterisk AMI `Originate`, Twilio REST API with an inline TwiML `<Say>` verb), 8kHz mono WAV conversion for telephony compatibility, and a three-tier delay-based escalation ladder. What it is missing, confirmed at Phase 0: DTMF acknowledgement handling, idempotency keys, and persisted `call_jobs` / `call_attempts` database records — the call is fired and logged, not tracked as a stateful job.

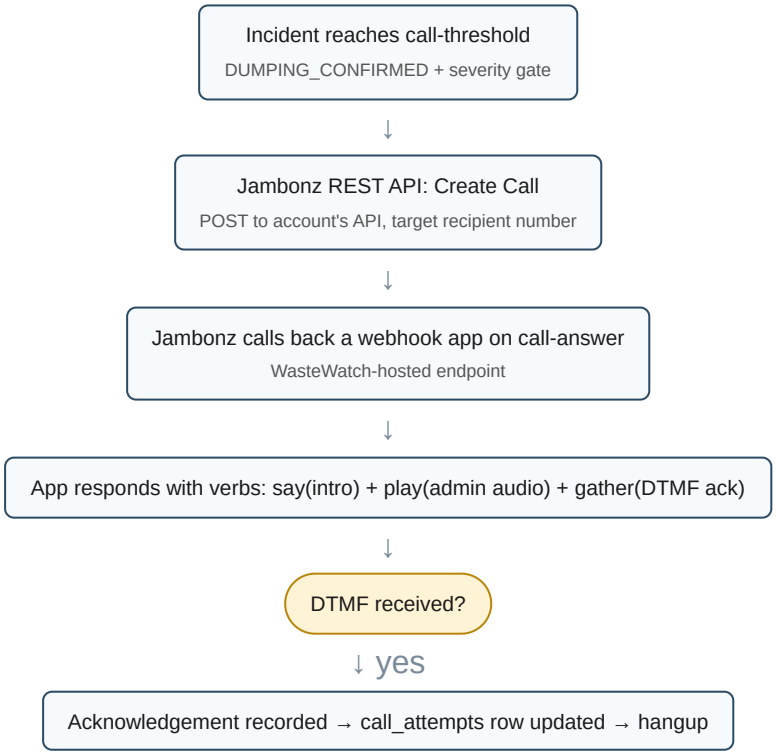
Jambonz as an additional or alternative backend

Jambonz is a self-hostable, open-source CPaaS (Communication Platform as a Service) — functionally comparable to Twilio, but with full control over the SIP/media stack (built on drachtio + FreeSWITCH/rtpengine components) and a programmable, webhook-driven call-control model instead of a closed proprietary API. An account has already been set up (SIP realm `satyakisolutions.sip.jambonz.cloud`, account SID and API key visible in the account's Settings page).

Why Jambonz fits the exact gap Phase 0 identified

> Jambonz's core model is a webhook-driven "application" that receives call events and responds with a JSON array of verbs (`say` , `play` , `gather` , `hangup` , etc.) — `gather` is specifically built for collecting DTMF or speech input mid-call. This maps directly onto the missing DTMF-acknowledgement requirement from Approach 2's spec, and does so more cleanly than bolting acknowledgement logic onto Twilio's inline-TwiML string-building approach or Asterisk's AMI-originate model currently in `sip_gateway.py` .

Proposed integration flow



Comparison of call-platform options

Option	Pros	Cons
Jambonz	Self-hostable, webhook/JSON call control, built-in DTMF gather primitive, already has an	Newer/smaller ecosystem than Twilio; requires hosting a webhook endpoint reachable from Jambonz's cloud or self-

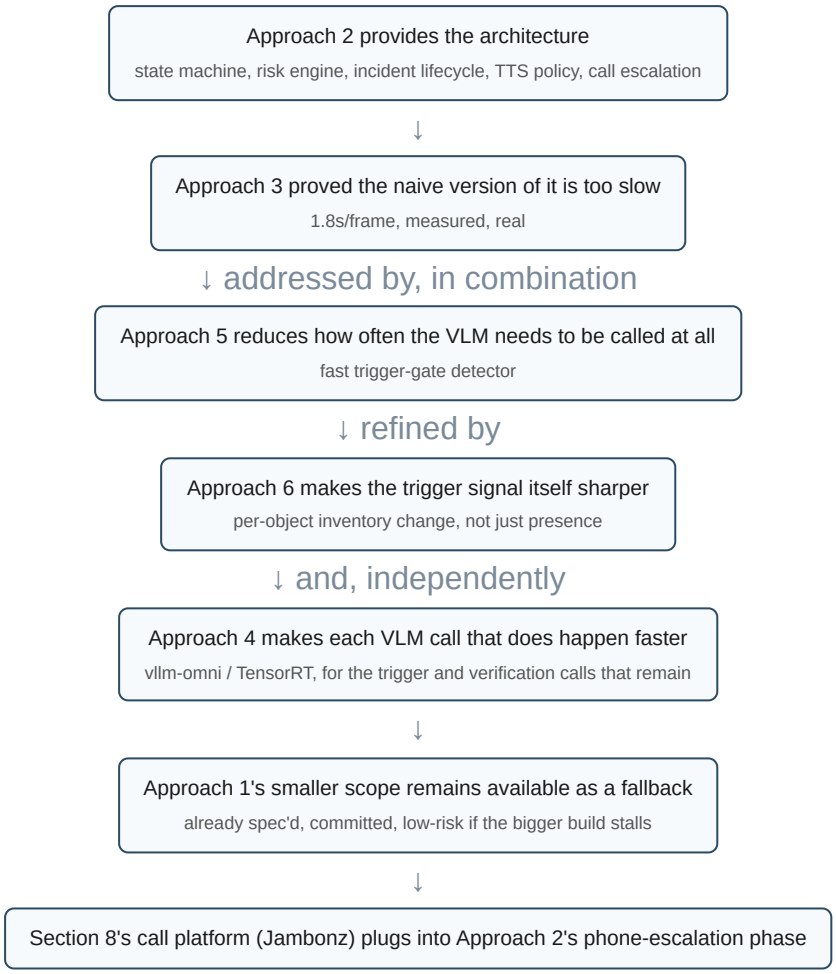
	account configured	hosted realm
Twilio (current, partial)	Already partially integrated in `sip_gateway.py`; mature, well-documented	Closed/proprietary, per-minute cost, TwiML string-building is less structured than Jambonz's JSON verb arrays for building ack logic
Asterisk AMI (current, partial)	Already partially integrated; fully self-hosted, no per-call cloud cost	AMI Originate is a lower-level primitive — DTMF/ack handling requires more custom dialplan work than a webhook-verb model
Fonoster	Open-source, gRPC-based, similar self-hosted positioning to Jambonz	Not currently integrated at all; would be a from-scratch addition with no existing account or code, unlike Jambonz which already has an account provisioned
VoIPBot (alyssaong1)	Lightweight, simple	Not currently integrated; smaller/community project, less suited to production call-acknowledgement workflows than Jambonz's purpose-built `gather` verb

Recommendation

Given an account is already provisioned and Jambonz's webhook/verb model directly addresses the DTMF-acknowledgement gap Phase 0 found in the existing partial implementation, Jambonz is the strongest candidate to become the primary call backend for Approach 2's phone-escalation phase — either replacing the Twilio/Asterisk paths in `sip_gateway.py`, or added as a fourth backend in the existing priority-ordered fallback chain. This has not been implemented; it is a recommendation based on capability fit, not a tested integration.

9. Synthesis: How the Six Approaches Compose Into One System

None of these six approaches are mutually exclusive alternatives — they are layers that stack, in the order they were discovered:



In other words: the target end-state is Approach 2's full architecture, with Approach 6's sharper per-object detection logic feeding Approach 5's fast trigger-gate, which calls a MiniCPM-o backend made fast enough by Approach 4's serving-engine work, escalating via Jambonz when confirmed — with Approach 1 sitting in reserve as a smaller, already-approved fallback scope if any of the above proves too large to complete.

10. Open Decisions

Decision	Status
Fold Approach 6's per-object tracking model into the committed design spec	Explicitly deferred by the user — held in context only, not yet written
Measure vllm-omni's real latency (Approach 4, Option C)	Not started
Decide on a fast trigger-gate model: zero-shot (YOLO-World) vs. custom-trained vs. distilled-from-MiniCPM-o	User indicated still thinking; install was paused mid-decision
Bag/object-carry labeled dataset sourcing	Not started — real blocker for any custom-trained option
Jambonz integration as primary vs. additional call backend	Recommended, not yet implemented or tested
Rewrite of the current aggressive/shaming TTS tone	Confirmed to do, not yet implemented
Object storage infrastructure for Approach 2's evidence/audio assets	Not started — new infrastructure, not a refactor
Event schema versioning convention across all Kafka producers	Not started — new pattern for this codebase

This document reflects the state of the investigation as of August 2026. It is a record of exploration and findings, not a finalized implementation plan — several of the approaches above (4, 5, 6) surfaced during real-time investigation and remain partially resolved by design.